

**Logique programmable pour systèmes
complexes et performants.
Ensemble de Julia**

Master of Science HES-SO in Engineering
Orientation: Electrical engineering (EIE)

Auteur

Groupe9: Malla Massimo, Nicoloso Matthias

Table des matières

1. Introduction	1
2. Simplification mathématique	2
2.1. Julia	2
2.2. Condition de sortie	2
3. Première implémentation	3
3.1. Machine d'état	3
3.2. Implémentation mémoire	4
3.3. Affichage HDMI	4
3.4. Résultat	5
4. Pipeline chaîné	6
4.1. Implémentation	6
4.2. Résultats:	7
5. Parallélisation	8
5.1. Parallélisation du pipeline	8
5.1.1. Résultat	8
5.2. Parallélisation spatiale	9
5.2.1. Diagramme de la machine d'état	9
5.2.2. Résultat	10
5.3. Un peu d'optimisation	10
5.3.1. Hiérarchie du calculateur	11
5.3.2. Liste des optimisations	11
6. Conclusion	12

Liste des figures

1	Ensemble de Julia groupe 9.	1
2	Machine d'état calcul Julia	3
3	Résultat Julia	5
4	Schéma fonctionnel pipeline chaîné	6
5	Schéma fonctionnel pipeline chaîné	7
6	Schéma fonctionnement pipeline parallélisé.	8
7	Schéma fonctionnement avec mémoire dédiée.	9
8	Schéma fonctionnement avec mémoire dédiée.	9
9	Rapport d'utilisation de l'implémentation finale.	10
10	Hiérarchie finale du calculateur.	11

1. Introduction

Le projet s'introduit dans le cours de logique programmable pour systèmes complexes et performants (LPSC).

L'objectif du projet est de mettre en place un calculateur de fractale de Mandelbrot ou un ensemble de Julia afin de l'afficher sur un écran HDMI de 720x720 pixels.

Le projet offrait 2 possibilités d'accélérateurs. La première était de faire communiquer 2 cartes via une interface série haute vitesse avec Aurora. La deuxième est de faire de l'accélération matérielle en optimisant la vitesse de calcul ainsi que de la parallélisation spatiale. Dans notre cas, nous avons choisi la 2ème solution.

Nous avons pour but d'implémenter l'ensemble de Julia suivant:

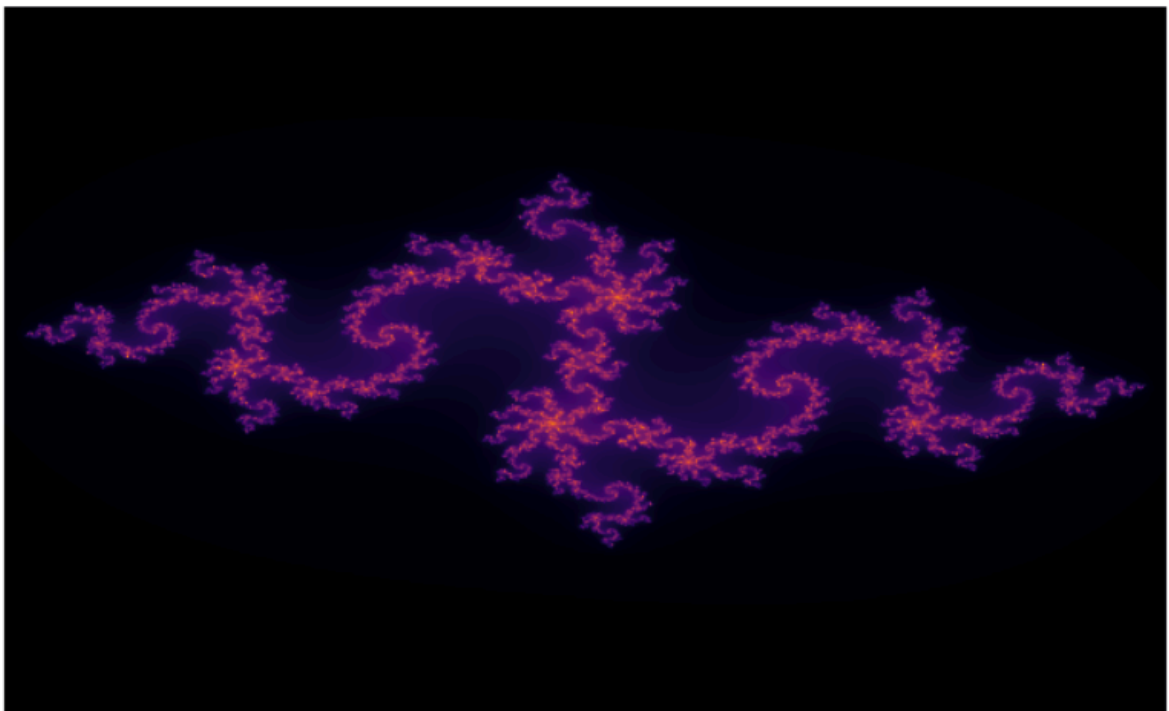


Figure 1. – Ensemble de Julia groupe 9.

La formule est la suivante:

$$z(z + 1) = z(n)^2 + c$$

avec $c = -0.835 - 0.232i$

2. Simplification mathématique

Avant de commencer la partie hardware, il est important de simplifier et de décomposer la formule mathématique afin d'optimiser les calculs.

2.1. Julia

Soit la formule de base:

$$z(z + 1) = z(n)^2 + c$$

en remplaçant par les parties réelles et imaginaires:

$$z(z + 1) = (x + y)^2 + a + i \cdot b$$

$$z(z + 1) = x^2 + 2 \cdot x \cdot y + (i \cdot y)^2 + a + i \cdot b$$

$$z(z + 1) = x^2 + 2 \cdot x \cdot y - y^2 + a + i \cdot b$$

Finalement en groupant les parties réelles et imaginaires:

$$z(z + 1) = x^2 + a - y^2 + i \cdot (2 \cdot x \cdot y + b)$$

avec x, y la position en pixel de l'ensemble et a, b la constante de Julia.

2.2. Condition de sortie

Ce calcul est effectué pour un total de 100 itérations maximum pour chaque pixel.

Il y a cependant une condition de sortie lorsque la norme dépasse 2:

$$2 > \sqrt{x^2 + y^2}$$

soit en simplifiant le calcul:

$$4 > x^2 + y^2$$

3. Première implémentation

3.1. Machine d'état

Notre première implémentation du système est naïve mais permet de comprendre comment ceci fonctionne avant de complexifier. Pour notre première implémentation nous avons décomposé les différentes parties du calcul en une machine d'état.

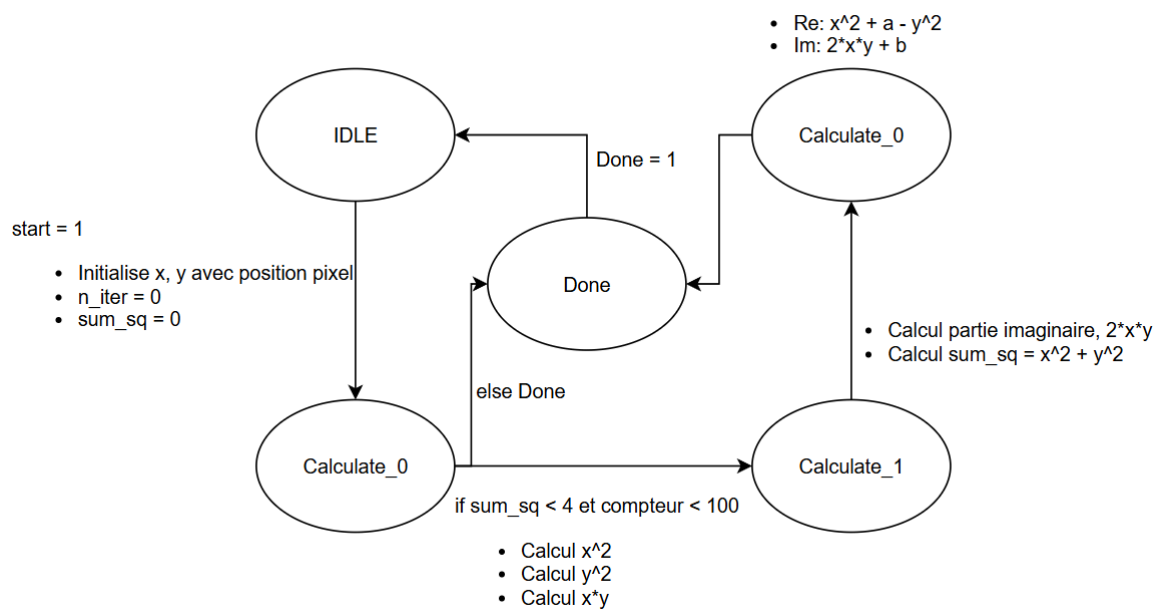


Figure 2. – Machine d'état calcul Julia

Comme le montre la figure 2, dans chaque état de « Calculate_X » une partie du calcul est effectuée. Cette manière de faire a le défaut d'augmenter la latence du résultat étant donné qu'il faut passer par plusieurs coups d'horloge avant le résultat final, mais a le grand avantage de pouvoir augmenter la vitesse d'horloge. En effet, étant donné que le calcul est séparé en plusieurs parties combinatoires, les chemins sont courts, permettant ainsi de pouvoir augmenter la fréquence du calcul.

3.2. Implémentation mémoire

Lorsque notre machine d'état a fini de calculer, la valeur du nombre d'itérations est stockée dans une mémoire. Pour ce faire, il a été important d'optimiser la taille de la mémoire étant donné que nous sommes limités par le hardware. Notre carte FPGA est capable d'avoir une mémoire allant jusqu'à environ 3 Mbits.

Dans notre cas, nous avons décidé d'utiliser une seule mémoire pour stocker toutes les itérations de chaque pixel. Sauf que nous avons rencontré un problème à ce niveau. Lorsque l'on calcule la largeur en bits de chaque case mémoire:

$$\text{bit width} = \frac{\text{nombre mémoire}}{\text{taille écran}} = \frac{3 \cdot 10^6}{720 \cdot 720} = 5.78$$

Cela signifie que nous avons à disposition 5 bits de stockage pour chaque pixel d'écran, or nous faisons un maximum de 100 itérations, soit 7 bits. Afin de régler ce problème, nous avons décidé de mettre en place une LUT afin de convertir les 7 bits en 5 bits, perdant ainsi un peu en résolution, mais la différence reste très faible à l'œil nu.

3.3. Affichage HDMI

Les données sont écrites dans la mémoire dans l'ordre, en partant de 0 jusqu'à la dernière case mémoire. Dans chaque case mémoire se trouve le nombre d'itérations sur 5 bits que le HDMI lit, sauf que ce dernier a besoin de 24 bits séparés entre R/G/B afin d'afficher un pixel. Pour cela il est nécessaire de faire une dernière conversion à l'aide d'une LUT afin de donner des frames correctes au HDMI. Nous avons donc créé une palette de couleurs de 5 bits, avec chacun des bits représentant une couleur en 24 bits.

3.4. Résultat

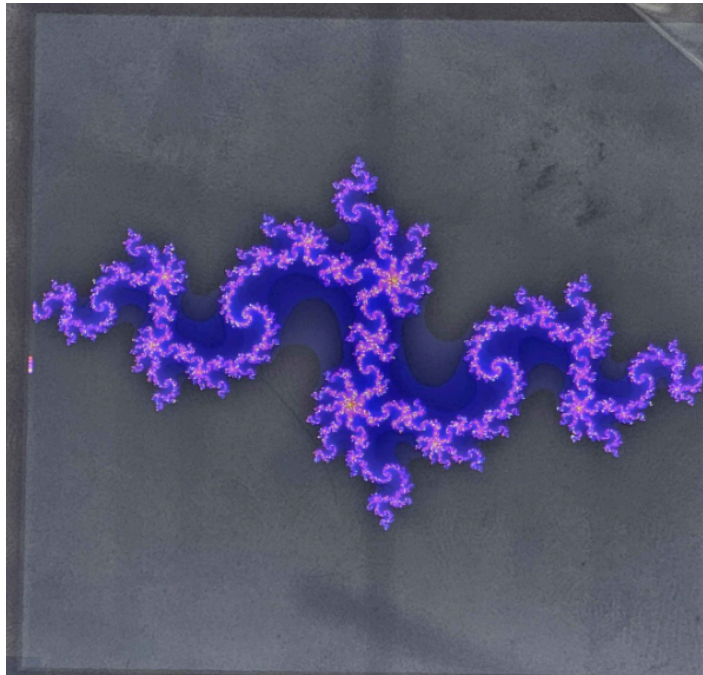


Figure 3. – Résultat Julia

Avec cette implémentation, nous utilisons 3 DSP, ceci est ce à quoi nous nous attendions étant donné que nous avons 3 multiplications dans le calcul de Julia, le reste étant un décalage ou des additions/soustractions.

En utilisant des VIO, il est possible de connaître le nombre d'itérations nécessaire afin de calculer l'ensemble. Pour cette première version, nous avons un total de 0xE45CBE itérations, soit 14 965 950 en décimal. En prenant en compte la taille de l'écran, nous pouvons calculer le nombre d'itérations moyen:

$$\frac{14965950}{720 \cdot 720} = 28.86 \text{ itérations}$$

En prenant en compte une vitesse d'horloge de 100 MHz, on peut calculer la vitesse de calcul:

$$\frac{100 \cdot 10^6}{14965950} = 6.68 \text{ fps}$$

Évidemment, cette fréquence et ce nombre d'itérations moyen sont des valeurs dynamiques, celles-ci vont changer lorsque l'on zoome. Mais cela reste néanmoins une bonne valeur de référence afin de comparer avec les prochains algorithmes.

4. Pipeline chaîné

4.1. Implémentation

Une grande faiblesse de la machine d'état précédente est que l'on calcule un pixel à la fois, ceci fait qu'on a une très grande latence entre chaque affichage à l'écran. Une manière d'optimiser est de faire un pipeline chaîné permettant ainsi de calculer plusieurs pixels en même temps. Par exemple, lors de la précédente implémentation avec la machine d'état, on calculait une partie après l'autre. Afin d'optimiser cela en un pipeline chaîné, il faut faire les 3 états de calculs en même temps pour 3 pixels différents. Afin d'implémenter ce pipeline chaîné, nous avons dû enlever notre machine d'état qui ne fonctionne pas pour ce genre de cas. La figure suivante illustre comment l'implémentation a été faite:

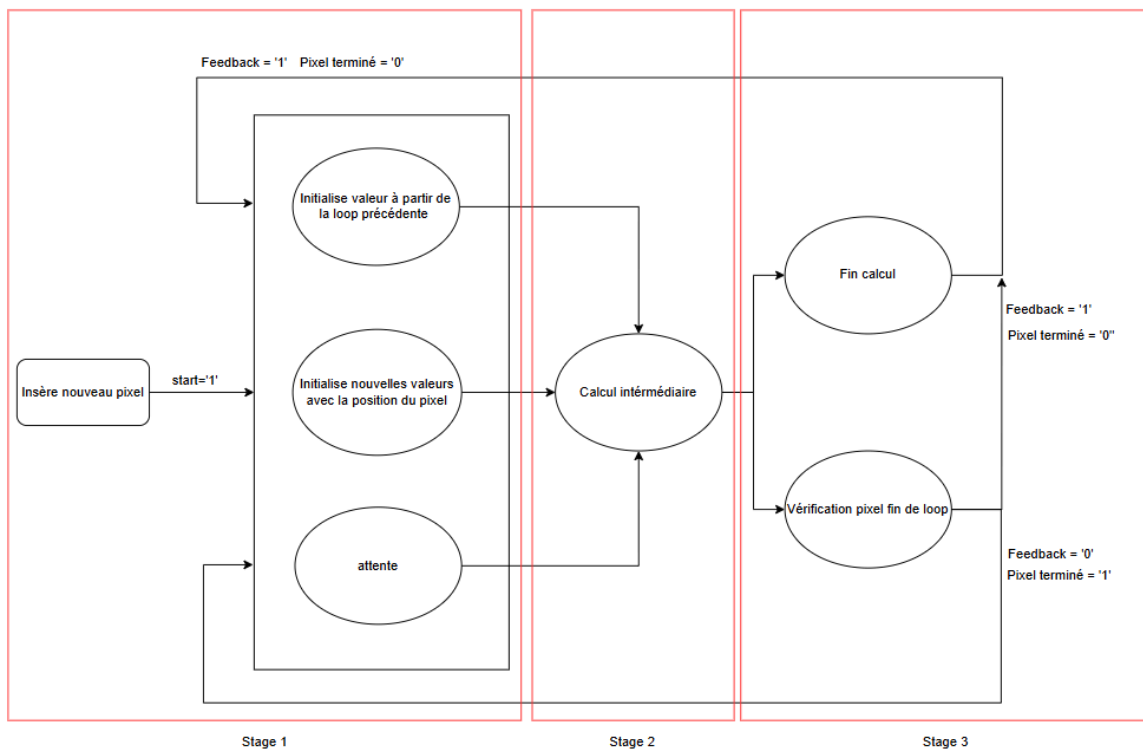


Figure 4. – Schéma fonctionnel pipeline chaîné

4.1 Pipeline chaîné – Implémentation

Ce pipeline possède plusieurs avantages, le premier est évidemment de pouvoir calculer plusieurs pixels en même temps, le deuxième est la possibilité d'augmenter la fréquence du système. Étant donné que l'on rajoute des bascules à la fin de chaque calcul, on peut aisément, si les conditions le permettent, augmenter la fréquence d'horloge. Il en est de même pour la machine d'état précédente.

Dans notre code, le calcul de Julia pipeliné peut être vu comme 5 étages (ou « stages »).

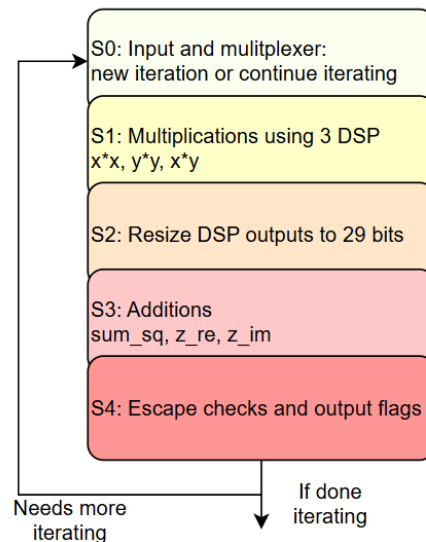


Figure 5. – Schéma fonctionnel pipeline chaîné

4.2. Résultats:

En appliquant la même technique que précédemment en utilisant un VIO, nous obtenons: 6'441'794 itérations. Cela correspond approximativement à nos prévisions. En faisant 3 pixels à la fois, nous augmentons la vitesse de x2.3 comparé à avant avec 14'965'950 itérations. Théoriquement, on devrait s'attendre à une augmentation de vitesse de x3, mais il y a des états intermédiaires dans lesquels il faut notifier que l'on a un espace libre, puis charger le pixel. Ceci rajoute 2-3 coups d'horloge. Plus on zoome sur la figure, moins ce phénomène aura d'impact car on aura un calcul d'itérations par pixel beaucoup plus grand. Lorsque l'on dézoome, la grande majorité de l'écran est noire, ce qui fait que ces états intermédiaires ont plus de conséquences.

DSP	vio hexa	vio dec	FPS
6	62 4B42	6 441 794	$\frac{100 \cdot 10^6}{6441794} = 15.52 \text{ fps}$

5. Parallélisation

5.1. Parallélisation du pipeline

Notre prochaine amélioration a été de paralléliser le pipeline chaîné afin d'augmenter la capacité de calcul. On peut monter jusqu'à 100 itérations dans les calculs de Julia, cela signifie que l'on n'écrit pas tout le temps dans la mémoire. Afin de profiter de ces moments dans lesquels la mémoire est en attente, on peut paralléliser le pipeline afin de permettre d'écrire dans la mémoire plus souvent. Il y a cependant un bottleneck principal à cette implémentation. Lorsque l'on calcule des zones en dehors de Julia, les calculs sont rapides et la mémoire actuelle ne permet pas d'écrire en même temps les différents résultats. Ce qui limite la parallélisation sur une grande partie de l'ensemble. Cependant, plus on zoome, plus ce bottleneck deviendra négligeable par le manque de zones noires selon la zone.

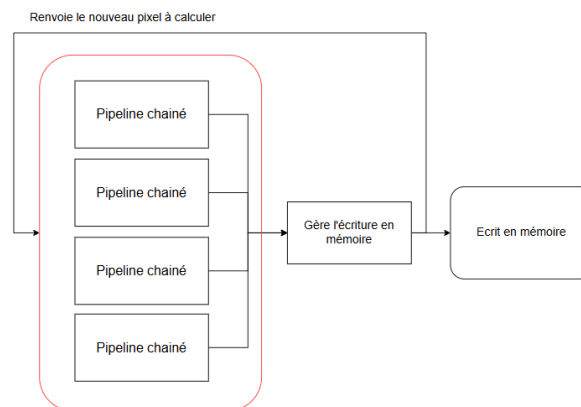


Figure 6. – Schéma fonctionnement pipeline parallélisé.

5.1.1. Résultat

DSP	vio hexa	vio dec	FPS
27	35 3550	3 487 056	$\frac{100 \cdot 10^6}{3487056} = 28.67 \text{ fps}$

5.2. Parallélisation spatiale

Afin d'optimiser l'écriture en mémoire, il est nécessaire de la séparer en plusieurs blocs. Actuellement, nous avons un gros bloc mémoire dans lequel on stocke toute l'image. Afin d'optimiser l'écriture en mémoire, il faut la séparer en plus petits blocs. Pour notre projet, nous avons décidé de séparer la zone mémoire en 4 blocs. Nous avons choisi 4 blocs par limitation du nombre de DSP disponibles dans le FPGA. Chacun de ces blocs mémoires est par la suite piloté par la parallélisation des pipelines, permettant ainsi d'avoir une très bonne optimisation.

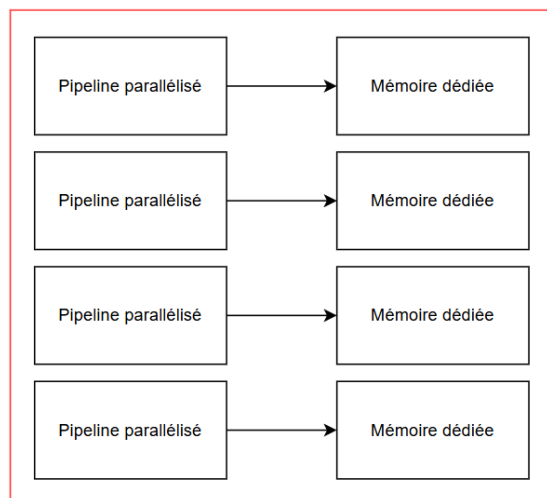


Figure 7. – Schéma fonctionnement avec mémoire dédiée.

5.2.1. Diagramme de la machine d'état

La machine d'état permet de parcourir les pixels qui définissent une région. L'écran de 720x720 pixels est séparé en 4 régions de 180x720 pixels.

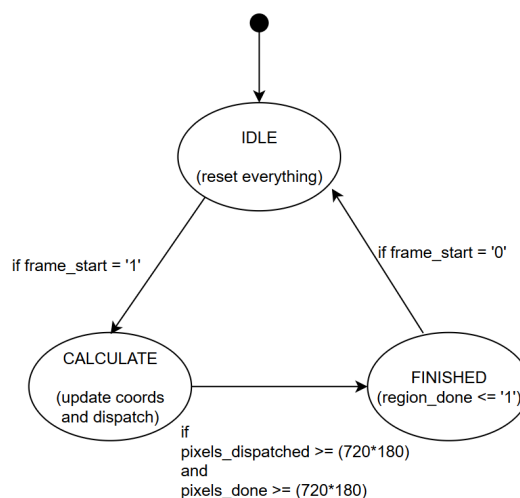


Figure 8. – Schéma fonctionnement avec mémoire dédiée.

5.2.2. Résultat

DSP	vio hexa	vio dec	FPS
101	15 87BC	1 411 004	$\frac{100 \cdot 10^6}{1411004} = 70.87 \text{ fps}$

5.3. Un peu d'optimisation

Actuellement, les tailles des bus sont de (3, -15), cette taille est parfaite car elle est de 18 bits, ce qui correspond exactement aux entrées des DSP. Ceci a permis d'obtenir de très bonnes performances sur les calculs, comme le montre le dernier résultat avec seulement 101 DSPs utilisés sur 160 disponibles. Cependant, cette optimisation est au détriment de la résolution, dû au faible nombre après la virgule disponible. Par la suite, la taille des bus de calcul de Julia a été augmentée à (3, -25) sauf que le nombre de DSP nécessaire était trop élevé pour notre calculateur. À ce moment-là, nous avons 4 mémoires séparées qui travaillaient en parallèle, et sur chacune des mémoires 4 autres parallélisations qui écrivaient dans la même mémoire par intervalles. Ce dernier a dû être baissé à 3 afin d'avoir assez de DSP pour un final de 144 DSP utilisés.

Nous avons essayé de faire une plus grande parallélisation sur la mémoire, en mettant 12 pipelines en parallèle mais avec 1 pipeline par mémoire à cause du nombre limité de DSP disponibles. Cela donnait des résultats moins performants que de mettre 4 mémoires avec 4 pipelines en parallèle. Cet exemple montre qu'il y a un juste milieu à trouver afin d'avoir les meilleures performances.

Name	Slice LUTs (46200)	Slice Registers (92400)	F7 Muxes (23100)	Slice (11550)	LUT as Logic (46200)	LUT as Memory (14400)	Block RAM Tile (95)	DSPs (160)	Bonded IOB (150)
scalp_user_design	13737	15054	10	5831	13405	332	80.5	144	30
> dbg_hub (dbg_hub)	473	753	0	242	449	24	0	0	0
> PLxB.gen_regions[0].region_inst (JuliaRegion)	2387	2576	1	1098	2345	42	0	36	0
> PLxB.gen_regions[1].region_inst (JuliaRegion_2)	2377	2593	1	1050	2335	42	0	36	0
> PLxB.gen_regions[2].region_inst (JuliaRegion_3)	2368	2593	1	1078	2326	42	0	36	0
> PLxB.gen_regions[3].region_inst (JuliaRegion_4)	2378	2552	1	1055	2336	42	0	36	0
> PLxB.HdmixB.Tx0d (scalp_hdmi)	252	77	0	87	252	0	0	0	0

Figure 9. – Rapport d'utilisation de l'implémentation finale.

5.3.1. Hiérarchie du calculateur

La version finale de notre calculateur est représentée comme suit. 4 instances de JuliaRegion, chacune ayant 3 instances du calculateur JuliaPipelined, après parallélisation.

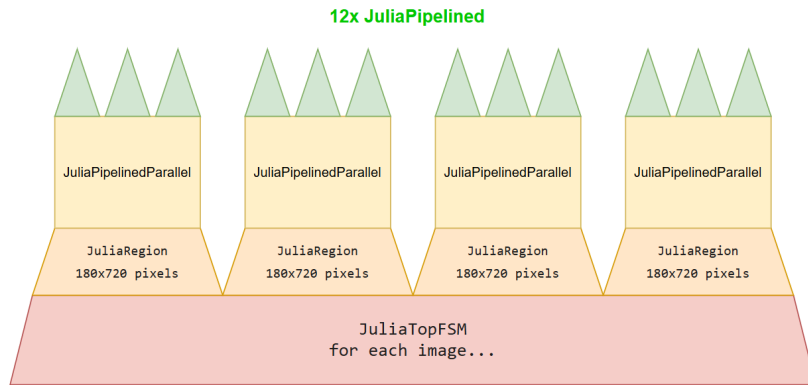


Figure 10. – Hiérarchie finale du calculateur.

5.3.2. Liste des optimisations

Les optimisations suivantes ont été mises en place afin d'assurer l'exécution la plus rapide possible, et en prenant compte de ce que notre matériel avait à disposition.

- Parallélisme spatial en découpant notre écran de 720x720 pixels en 4 régions de 180x720 pixels
- Parallélisation des workers, chaque région a trois instances de JuliaPipelined, avec un dispatcher pour l'assignation de workers
- Ajout d'attributs « use_dsp » pour forcer l'utilisation de DSPs
- Ajout d'attributs « dont_touch » pour forcer les multiplications à deux cycles
- Utilisation de LUT dans la ROM, plutôt qu'utiliser de la BRAM
- Écriture dans BRAM seulement après flag « jpp_done »
- Utilisation de FIFO dans les collecteurs au cas où 2 workers termineraient au même cycle

6. Conclusion

La séquence de calcul pour l'animation du zoom est d'environ 17s. Cette mesure a été faite en filmant la séquence et en prenant le delta entre le début et la fin. En prenant en compte que la fenêtre de départ est de 3x3 et que le zoom est jusqu'à une fenêtre de 0.05 avec des steps de 0.01, il y a un total d'images de:

$$\frac{3 - 0.05}{0.01} = 295 \text{ images}$$

soit une fréquence de calcul moyenne:

$$\frac{295}{17} = 17.35 \text{ fps}$$

Avec la version de calcul 18 bits avec moins de résolution, le résultat était de 13s pour la séquence soit:

$$\frac{295}{13} = 22.69 \text{ fps}$$

Cela fait une différence de 24%. Ceci s'explique par le fait que lorsque la résolution est plus haute, Vivado doit chaîner plusieurs DSP ensemble, forçant ainsi à ralentir le design pour garantir l'arrivée des signaux. Cependant dans notre cas, grâce au pipeline, l'impact de ce phénomène est minime. La différence provient du fait que lorsque l'on zoome, la densité de calculs complexes augmente, ce qui nécessite des itérations supplémentaires avec pour conséquence de ralentir globalement le système. Le nombre final de DSPs utilisés est de 144 sur 160 disponibles. Notre implémentation permet ainsi d'utiliser toute la puissance de calcul disponible dans le FPGA. n permet ainsi d'utiliser toute la puissance de calcul disponible dans la FPGA.